Self-Reflection

Youyang Zhao

Description

What happened during the project?

During this project, I developed GoReady, a weather-based utility app that helps users decide what to prepare before going outside. The app was built around one simple question: “What should I prepare before leaving?” Instead of creating a full weather forecast app, I focused on four weather factors that directly affect daily outdoor preparation: temperature, rain chance, UV index, and wind speed. These values are converted into practical advice, such as bringing an umbrella, using sunscreen, drinking more water, wearing an extra layer, being careful in strong wind, or being ready to go.

To build the app, I used Kotlin, Jetpack Compose, Material Design 3, ViewModel, Repository pattern, Retrofit, UI state management, simple dependency injection, reusable Compose components, and Android MediaPlayer. The final app includes a Utility screen, a Settings screen, live weather data from the Open-Meteo API, weather-based advice, dynamic weather icons, animated advice visuals, weather detail gauges, manual refresh, and optional background music.

Feelings: What were my feelings/reactions throughout the project?

What were my feelings at the beginning of the project?

At the beginning, I felt confident about the app idea because it was practical and easy to understand. A utility app should help users complete a daily task quickly, and GoReady had a clear everyday use case. I liked the idea that the app could give users a direct suggestion instead of making them read a full weather report.

At the same time, I was not fully confident about turning the idea into a complete Android app. Building the first Compose layout was not too difficult, but the project became more challenging when I needed to connect the screen with live weather data, ViewModel state, repository logic, Retrofit API calls, settings controls, and animated components. I realized that the app needed more than a good-looking interface. It also needed a clear internal structure.

How did my feelings change during development?

As the project developed, I became more confident because the app changed from a screen design into something that actually worked. When the app successfully loaded live weather data from the Open-Meteo API and showed it on the Utility screen, the

project felt much more real. I could see the repository getting data, the ViewModel updating the state, and the Compose UI showing the latest information.

I also felt more confident when the Settings screen started to control the Utility screen properly. Changing the city, temperature unit, detail visibility, advice length, layout, or background music setting all affected the main screen. This helped me understand that the Settings screen was connected to the main app logic, not just an extra page.

What reactions did I have when I faced challenges?

When I faced challenges, I often felt frustrated at first. Some changes looked small but affected more parts of the app than I expected. For example, changing the advice rule meant checking whether the advice text, hero icon, animated visual card, and weather status still matched each other. This made me realize that the app was more connected than I first thought.

Later, this became useful because it helped me understand why app structure matters. After I separated the weather data, advice logic, UI state, and reusable components more clearly, I felt less lost when making changes.

Evaluation: What went well and what didn't go well?

What went well?

One thing that went well was the app purpose. I kept GoReady focused on quick outdoor preparation instead of adding too many weather features. This made the app more suitable as a utility app because users can quickly see the city, temperature, weather status, advice, visual feedback, and optional detail gauges.

The architecture also worked well after I separated the responsibilities of different parts of the app. The ViewModel manages selected city, weather data, loading state, error message, and user settings. The repository handles weather data access. Retrofit and the API service handle the Open-Meteo request. The helper functions convert weather values into advice and status text. This made the code clearer and prevented the UI from doing too much work.

The Settings screen also worked well because it directly controls the Utility screen. It changes the selected city, temperature unit, detail visibility, advice detail level, advice card layout, and background music setting. This made the Settings screen meaningful rather than decorative.

What didn't go well?

One thing that did not go well at first was my development order. I spent a lot of time improving the visual design, such as cards, gradients, icons, animations, and layout details. These changes made the app look better, but they did not automatically make the code easier to manage. Later, I had to revise the structure to make the data flow clearer.

Another challenge was the connection between the Settings screen and the Utility screen. At first, I thought the settings would be simple because they were only switches and city selections. However, each setting affected different parts of the app. This helped me understand that settings should be connected to shared state, not handled as isolated UI controls.

What was the most meaningful experience from this project?

The most meaningful experience was seeing GoReady move from a visual idea into a working Android app. At first, I mainly focused on the interface. Later, the app included live API data, state management, advice logic, interactive settings, animated feedback, and optional background music. This helped me understand that Android coding is not only about making screens. It is also about how data moves through the app and how state controls the UI.

Analysis: Why did things go well or not well?

Why did things go well?

The project went well because the app had a clear purpose from the beginning. Since GoReady focused on outdoor preparation, I could choose weather data that supported this purpose directly. Temperature, rain chance, UV index, and wind speed were enough to generate useful advice without making the app too complicated.

The project also improved when the code responsibilities became clearer. Once the ViewModel, repository, network layer, helper functions, and UI components had separate roles, the app became easier to understand and control. This also helped the Settings screen work more predictably because changes were handled through GoReadyUiState and the ViewModel.

Reusable components also helped. Separating cards, gauges, icons, city selection, setting rows, and visual advice sections made the interface easier to adjust and reduced repeated code.

Why did some things not go well at first?

Some things did not go well because I underestimated how connected the app would become. I first focused on what the user could see, such as the layout and visuals. Later, I

had to think more about how data, state, and logic moved through the app. When I changed advice rules or layout, I sometimes needed to check several files. This showed me that a good-looking UI still needs clear logic behind it.

The Settings screen was also more complex than I expected. Changing one setting could affect the weather data, advice text, icon, visual card, detail gauges, layout, or background music. This helped me understand why shared state is important in Jetpack Compose.

Conclusion

What did I learn?

Through this project, I learned that a strong Android app needs both a clear purpose and a clear structure. GoReady became better when I stopped thinking only about the interface and started thinking about data flow, state management, and separation of responsibilities.

I also learned that ViewModel, Repository pattern, Retrofit, domain helper functions, and reusable Compose components are not just technical requirements. They make the app easier to build, debug, explain, and improve. The project also showed me that every feature should support the main purpose. For GoReady, the live weather data, advice rules, settings, visual feedback, and manual refresh all support quick outdoor preparation.

Action Plan

What will I do next time?

Next time I work on a similar Android project, I will plan the application logic first and then spend too much time on visual design. Next time, I will create a simple plan first and then start writing too much UI code. I will first figure out what information the app needs, where this information comes from, and which page will use this information.

To make GoReady more usable, I will add location-based real-time weather forecasts, so users can get suggestions based on their current location instead of being limited to manually selecting a city. I'll also add a short checklist of items to prepare based on the suggestions, such as an umbrella, sunscreen, water bottle, or light jacket, and more clearly display the "last updated" time after refreshing the weather data. Finally, I'll test the main suggestion rules in advance to ensure the app provides the correct suggestions before finalizing the user interface.